MANIPAL INSTITUTE OF TECHNOLOGY

BENGALURU

(A constituent unit of MAHE, Manipal)

School of Computer Science and Engineering

CSS_2213-Object-Oriented Software Development Lab

Project Report - JavaFX and collections

Academic Year: 2025-26 **Semester: IV**

Branch: IT **Section: B**

Name of the Application: Library Management System

Student Name: Savya Raj

Register Number: 245811454

Under the guidance

S. Vaishnavi

Ms. M. R. Zeba Rehaman

Ms. Shree Lakshmi Yadav

Table of Contents

1. Abstract
2. System Requirements
   - 2.1 Hardware Requirements
   - 2.2 Software Requirements
3. Objective
4. Tools & Technologies
   - 4.1 Programming Language — Java 25
   - 4.2 GUI Framework — JavaFX 23.0.1
   - 4.3 Build Tool — Apache Maven 3.x
   - 4.4 IDE — IntelliJ IDEA 2025.3
5. System Description
   - 5.1 Dashboard Tab
   - 5.2 Rooms Tab
   - 5.3 Customers Tab
   - 5.4 Bookings Tab
6. Implementation Details
   - 6.1 Modules
   - 6.2 Use of Collections
7. GUI Design
   - 7.1 Overall Layout
   - 7.2 Status Badges
   - 7.3 Button Styling
   - 7.4 Input Validation
8. Program Structure
9. Conclusion
10. References
11. Appendix (Code Snippets)
    - A1 Module Declaration (module-info.java)
    - A2 Room Class (BooleanProperty Implementation)
    - A3 RoomManager (ObservableList & FilteredList)
    - A4 BookingManager (Booking Method)
    - A5 BookingManager (Checkout & Bill Generation)
    - A6 BookingView (DatePicker Implementation)
    - A7 BookingView (FilteredList for Active Bookings)
    - A8 DashboardView (ListChangeListener & Refresh Logic)

1. Abstract

HotelDesk is a desktop-based Hotel Management System developed using Java 25 and the JavaFX 23 UI framework. The application provides an integrated platform for hotel staff to manage rooms, register guests, create date-aware bookings with explicit check-in and check-out dates, and process checkouts with automatic bill generation.

The system is structured in three distinct layers: a model layer of plain Java POJOs backed by JavaFX observable properties, a manager layer that owns all business logic and exposes ObservableList collections, and a view layer that builds the JavaFX scene graph and binds directly to those collections for automatic UI refresh. Navigation is implemented using a JavaFX TabPane with four tabs — Dashboard, Rooms, Customers, and Bookings — each rendered as a white card panel over a light blue-grey (#f0f4f8) background.

The project demonstrates practical use of JavaFX observable properties (BooleanProperty, IntegerProperty), ObservableList extractors, FilteredList predicates, DatePicker controls, lambda event handlers, the Java Platform Module System, and Apache Maven as a build tool.

2. System Requirements

2.1 Hardware Requirements

Component	Minimum Specification
Processor	Intel Core i3 or equivalent, 1.8 GHz or higher
RAM	4 GB minimum (8 GB recommended for smooth JavaFX rendering)
Storage	100 MB free disk space for project and JDK
Display	1280 × 720 px minimum resolution (Full HD recommended)
Input	Standard keyboard and mouse

2.2 Software Requirements

Software	Version	Purpose
Java Development Kit	JDK 25 (Temurin / OpenJDK)	Runtime and compilation
JavaFX (OpenJFX)	23.0.1	GUI framework (declared in pom.xml)
Apache Maven	3.x (bundled via mvnw)	Build tool and dependency management
javafx-maven-plugin	0.0.8	Provides mvn javafx:run goal
IntelliJ IDEA	2024.x	Recommended development IDE
Operating System	Windows 10/11, macOS 12+, or Linux	Any OS supporting JDK 25

3. Objective

The primary objective of this project is to design and implement a fully functional, self-contained desktop Hotel Management System that demonstrates the practical integration of advanced Java programming concepts with a JavaFX graphical user interface.

The specific objectives are:

1. To implement a modular Java application using the Java Platform Module System (JPMS) with a single module declaration in module-info.java.
2. To apply object-oriented design principles — encapsulation, abstraction, and separation of concerns — across the model, manager, and view layers.
3. To use JavaFX observable properties (BooleanProperty on Room, IntegerProperty on Customer) with ObservableList extractors to propagate fine-grained property changes to the UI automatically.
4. To implement date-aware bookings using Java's LocalDate and DatePicker controls, with explicit check-in and check-out date selection and validation.
5. To use FilteredList to enable dynamic room-availability filtering without duplicating the underlying data collection.
6. To support the four core hotel operations: room inventory management, guest registration, room booking (with dates), and checkout with formatted bill generation.
7. To deliver a fully runnable Maven project that requires no external database, making it self-contained and straightforward to build and evaluate.

4. Tools & Technologies

4.1 Programming Language — Java 25

The entire application is written in Java 25 (OpenJDK). The pom.xml declares maven.compiler.source and maven.compiler.target both as 25. Language features used include: lambda expressions for event handlers (e.g. saveBtn.setOnAction(e -> {...})) and ObservableList listeners, method references (Room::isAvailable, Booking::isActive), enhanced for-each loops in manager lookup methods, and String.format() for the bill summary. The Java Platform Module System (JPMS) is used for module encapsulation.

4.2 GUI Framework — JavaFX 23.0.1

JavaFX (OpenJFX 23.0.1) is the GUI framework. The dependency javafx-controls 23.0.1 is declared in pom.xml; it transitively includes javafx-graphics and javafx-base. The following JavaFX types are directly used across the four views:

JavaFX Class / Type	Usage in HotelDesk
TabPane	Root navigation — four non-closable tabs (Dashboard, Rooms, Customers, Bookings)
Tab	Each view returns a Tab from its createTab() method; set non-closable and 13 px font
BorderPane	Root scene layout: HBox header at top (setTop), TabPane in centre (setCenter)
HBox	Header bar, dashboard card rows, filter button bar, form field rows
VBox	Primary layout inside each tab; card containers; left/right panels
TableView<T>	Room, Customer, and Booking tables with cell factories
TableColumn<S,T>	Columns with PropertyValueFactory bindings and custom CellFactory for formatting
ComboBox<T>	Customer, room, and active-booking selection dropdowns
DatePicker	Check-in and check-out date selection in BookingView (defaults: today / today+1)
TextField	Room number, price, customer name, and contact number inputs
Button	Save Room, Register Customer, Book Now, Checkout action triggers
Label	Dashboard stat values, section headings, feedback messages, badge pills
Alert / TextArea	Validation alerts (ERROR, WARNING, INFORMATION) and the checkout bill dialog
Separator	Visual dividers inside card panels between sections
ScrollPane (none)	Not used — content fits within the fixed 1280×720 window area
BooleanProperty	Room.available field; extractor exposes it to ObservableList change detection
IntegerProperty	Customer.assignedRoom field; extractor exposes it to ObservableList
FilteredList<Room>	RoomManager wraps rooms list; predicate toggled by Available Only / View All buttons
FilteredList<Booking>	BookingView.refreshActiveBookings() filters bookings to populate the checkout ComboBox
ListChangeListener	Attached in DashboardView to both rooms and bookings lists to call refreshDashboard()

4.3 Build Tool — Apache Maven 3.x

Apache Maven manages the build lifecycle. The Maven Wrapper (mvnw / mvnw.cmd) is included so no local Maven installation is required. The pom.xml declares groupId com.hotel, artifactId HotelDesk, version 1.0.0. The only runtime dependency is javafx-controls 23.0.1 from org.openjfx. The javafx-maven-plugin 0.0.8 is configured with mainClass set to com.hotel.hoteldesk/com.hotel.hoteldesk.Main, following JPMS module/class syntax.

4.4 IDE — IntelliJ IDEA 2025.3

IntelliJ IDEA was used for development, providing first-class Java 25, JavaFX, and Maven support. The project structure is standard Maven layout (src/main/java, src/main/resources) and opens directly via File → Open in IntelliJ without any additional configuration.

5. System Description

HotelDesk launches via the JavaFX Application lifecycle (Main extends Application). The start() method instantiates the three manager classes, passes them to the four view classes, and assembles the scene. The main window is a BorderPane: a styled gradient header HBox at the top (linear-gradient from #1a5276 to #2980b9), and a TabPane in the centre containing the four non-closable tabs. The window is fixed at a minimum of 1280 × 720 px. Six sample rooms are pre-loaded on startup by RoomManager.loadSampleRooms().

5.1 Dashboard Tab

DashboardView.createTab() returns a Tab whose content is a VBox (background #f0f4f8, 40 px padding). The view shows a bold title label, a subtitle, a Separator, then two HBox rows of stat cards. Each card is a VBox (white background, 12 px radius, coloured bottom border 4 px, Gaussian drop shadow) showing a muted title label and a large bold value label. The five stats are: Total Rooms, Available, Occupied, Active Bookings, and Total Revenue. ListChangeListeners are attached to both the rooms and bookings ObservableLists so that refreshDashboard() is called automatically on any change.

5.2 Rooms Tab

RoomView.createTab() returns a Tab with an HBox content area: a form panel (VBox, fixed 280 px width) on the left and a table panel (HBox.setHgrow ALWAYS) on the right. The form accepts Room Number (TextField, integer), Room Type (ComboBox pre-populated with SINGLE / DOUBLE / DELUXE from the RoomType enum, default SINGLE), and Price Per Day (TextField, double). Validation uses Alert dialogs (ERROR type). The table panel shows a TableView bound to RoomManager.getFilteredRooms() with four columns: Room #, Type, Price/Day (formatted Rs.%.2f), and Status (badge label with rounded pill style). Two buttons — Available Only and View All — toggle the FilteredList predicate.

5.3 Customers Tab

CustomerView.createTab() uses the same HBox split layout. The form registers a new guest with Full Name and Contact (10 digits). A textProperty listener on the contact field strips non-digit characters and enforces a 10-character maximum in real time. Customers are assigned sequential IDs starting from 1001 (nextId in CustomerManager). An Occupancy box (background #eaf2f8) in the form panel shows the live occupied/total ratio, updated by a ListChangeListener on the rooms list. The table binds to customerManager.getCustomers() and shows ID, Name, Contact, and Assigned Room (displayed as '-' when assignedRoom == -1, computed via a SimpleStringProperty lambda).

5.4 Bookings Tab

BookingView.createTab() renders an HBox with a left panel (fixed 340 px) and a right all-bookings table. The left panel contains two white card VBoxes stacked vertically. The Book a Room card has: a Customer ComboBox (bound to customerManager.getCustomers()), an Available Room ComboBox (populated on setOnShowing by filtering rooms with Room::isAvailable), a Check-in DatePicker (default: LocalDate.now()), and a Check-out DatePicker (default: LocalDate.now().plusDays(1)). The Book Now button validates that both selections and dates are present and that checkOut is not before checkIn, then calls bookingManager.book(customer, room, checkIn, checkOut). The Checkout card shows a bookingCombo (refreshed by refreshActiveBookings() which uses a FilteredList predicate Booking::isActive). Clicking Checkout calls bookingManager.checkout(selected) and displays the returned bill string in a TextArea inside an INFORMATION Alert dialog.

6. Implementation Details

6.1 Modules

The application is fully modularised under the Java Platform Module System. The module descriptor at src/main/java/module-info.java declares the module named com.hotel.hoteldesk:

```
module com.hotel.hoteldesk {
    requires javafx.controls;

    // Allows the JavaFX launcher to reflectively instantiate
    // Main (which extends Application) via javafx.graphics
    opens com.hotel.hoteldesk to javafx.graphics;

    // Allows PropertyValueFactory to call getters on model
    // objects via reflection (javafx.base uses reflection)
    opens com.hotel.hoteldesk.model to javafx.base;

    exports com.hotel.hoteldesk;
}
```

requires javafx.controls is the only module requirement. This transitively brings in javafx.graphics (scene graph, layout panes, CSS) and javafx.base (observable properties, ObservableList, FXCollections). The model package is opened to javafx.base specifically because PropertyValueFactory uses reflection to call getters such as getRoomNumber() and getAvailable() on model objects inside TableView columns. Without this open directive the application throws an InaccessibleObjectException at runtime.

The four packages that form the module are:

- com.hotel.hoteldesk — Main.java only. Assembles managers, views, header, TabPane, and root Scene.
- com.hotel.hoteldesk.model — Room, Customer, Booking (plain Java with JavaFX observable properties), RoomType enum.

- com.hotel.hoteldesk.manager — RoomManager, CustomerManager, BookingManager. Owns all ObservableLists and business rules.
- com.hotel.hoteldesk.view — DashboardView, RoomView, CustomerView, BookingView. Each exposes a createTab() method that returns a fully built Tab.

6.2 Use of Collections

HotelDesk uses JavaFX observable collections throughout to drive automatic, reactive UI updates. The following table documents every collection type, where it lives, and how it is used:

Collection	Owner	Purpose & Detail
ObservableList<Room>	RoomManager	Created via FXCollections.observableArrayList() with a property extractor (see below). Backing source for the rooms TableView and the FilteredList. Pre-populated with 6 sample rooms by loadSampleRooms().
ObservableList<Customer>	CustomerManager	Also created with a property extractor on assignedRoomProperty(). Bound directly to the Customers TableView via table.setItems() and to the Customer ComboBox in BookingView via customerCombo.setItems().
ObservableList<Booking>	BookingManager	Standard FXCollections.observableArrayList() (no extractor needed — bookings are replaced via bookings.set(idx, booking) on checkout to force a list-change event). Bound to the Bookings TableView.
FilteredList<Room>	RoomManager	Wraps the rooms ObservableList. Predicate defaults to p -> true (show all). RoomManager.setFilterAvailableOnly(true) switches it to Room::isAvailable; setFilterAvailableOnly(false) restores the all-rooms predicate. The RoomView TableView binds to getFilteredRooms().
FilteredList<Booking>	BookingView	Created inline inside refreshActiveBookings() via bookingManager.getBookings().filtered(Booking::isActive). The result is passed to bookingCombo.getItems().setAll() to populate the checkout dropdown with only active bookings.
ListChangeListener<Room>	DashboardView	Attached to roomManager.getRooms() inside createTab(). Fires refreshDashboard() on every structural change (add / remove) and every available property change (thanks to the extractor), keeping the Available and Occupied cards accurate.
ListChangeListener<Booking>	DashboardView	Attached to bookingManager.getBookings(). Fires refreshDashboard() on add or element replacement (checkout triggers bookings.set(idx, booking)). Keeps Active Bookings and Total Revenue cards accurate.
SimpleStringProperty	CustomerView	Used in the Assigned Room column cell-value factory: new SimpleStringProperty(room == -1 ? "-" : String.valueOf(room)). Converts the integer property to a display string without a full custom cell factory.

The ObservableList extractors are the most important collection pattern in the project. Both RoomManager and CustomerManager use them:

```
// RoomManager.java
rooms = FXCollections.observableArrayList(
    room -> new Observable[]{ room.availableProperty() }
);

// CustomerManager.java
customers = FXCollections.observableArrayList(
    customer -> new Observable[]{ customer.assignedRoomProperty() }
);
```

Without an extractor, modifying a property of an existing list element (e.g. setting room.setAvailable(false) during booking) would NOT fire a list-change event because the list itself has not structurally changed. The extractor lambda tells the ObservableList to subscribe to the specified property on each element, so any property change fires a list-update event. This is what keeps the Dashboard stats and the Room table status badges live without any explicit refresh calls.

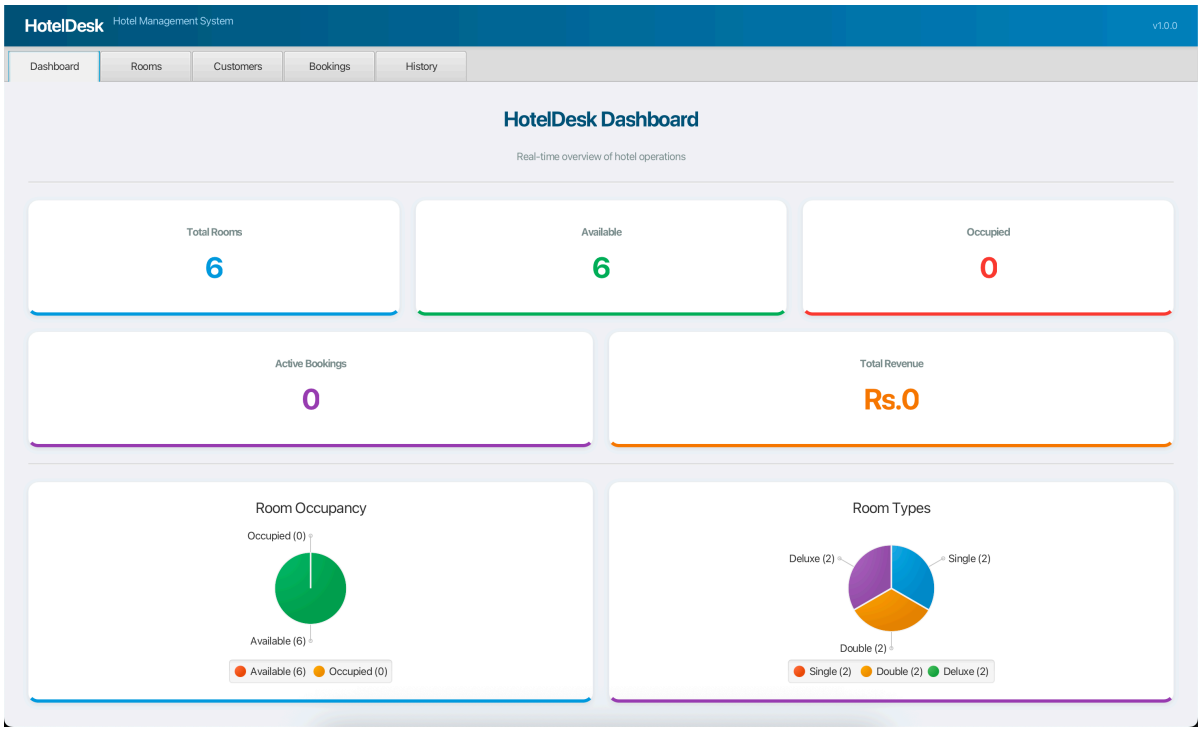
The Booking ObservableList does not use an extractor because the Booking object is immutable in structure after creation — checkout modifies its state and then the manager calls bookings.set(idx, booking) to replace the element in the list, which always fires a structural change event:

```
// BookingManager.java — checkout() method
int idx = bookings.indexOf(booking);
if (idx >= 0) {
    bookings.set(idx, booking);  // forces list-change event
}
```

7. GUI Design

7.1 Overall Layout

The root scene node is a BorderPane. setTop() holds a custom gradient header HBox (background: linear-gradient(to right, #1a5276, #2980b9), 12 px vertical padding, 25 px horizontal padding) containing the HotelDesk logo label (white, bold, 22 pt), a tagline label (#aed6f1, 13 pt), a flexible spacer Region, and a version label v1.0.0. setCenter() holds the TabPane. Tabs are set to a minimum width of 130 px and a minimum height of 36 px, with 13 px font size.

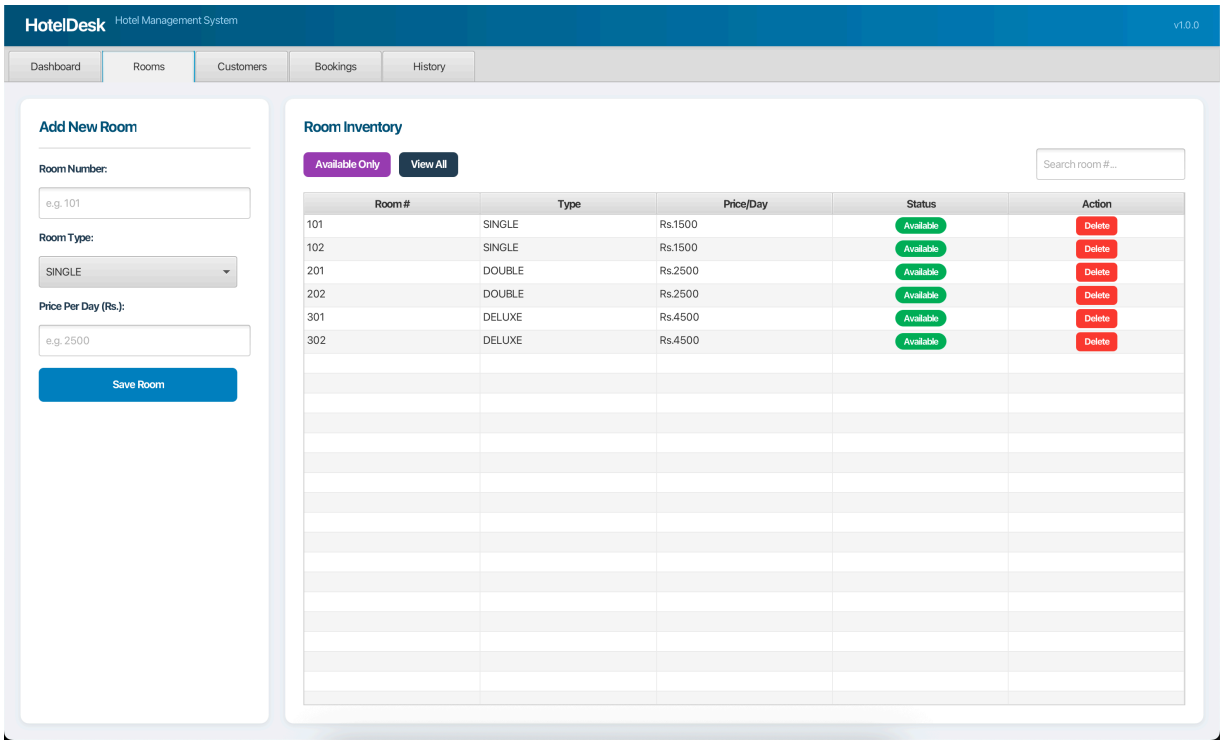


7.2 Status Badges

The Status column in the Rooms table uses a custom CellFactory. Each cell renders a Label pill badge (not plain text) with white foreground text, 12 px rounded corners, and 4/12 px padding:

```
Label badge = new Label(available ? "Available" : "Occupied");
badge.setPadding(new Insets(4, 12, 4, 12));
badge.setFont(Font.font("System", FontWeight.BOLD, 11));
badge.setTextFill(Color.WHITE);
badge.setStyle(available
    ? "-fx-background-color: #27ae60; -fx-background-radius: 12;"
    : "-fx-background-color: #e74c3c; -fx-background-radius: 12;");
```

The Bookings table Status column uses a PropertyValueFactory bound to getStatus() (which returns the String "Active" or "Checked Out") with a badge Label cell factory applying #3498db for active and #95a5a6 for checked out.

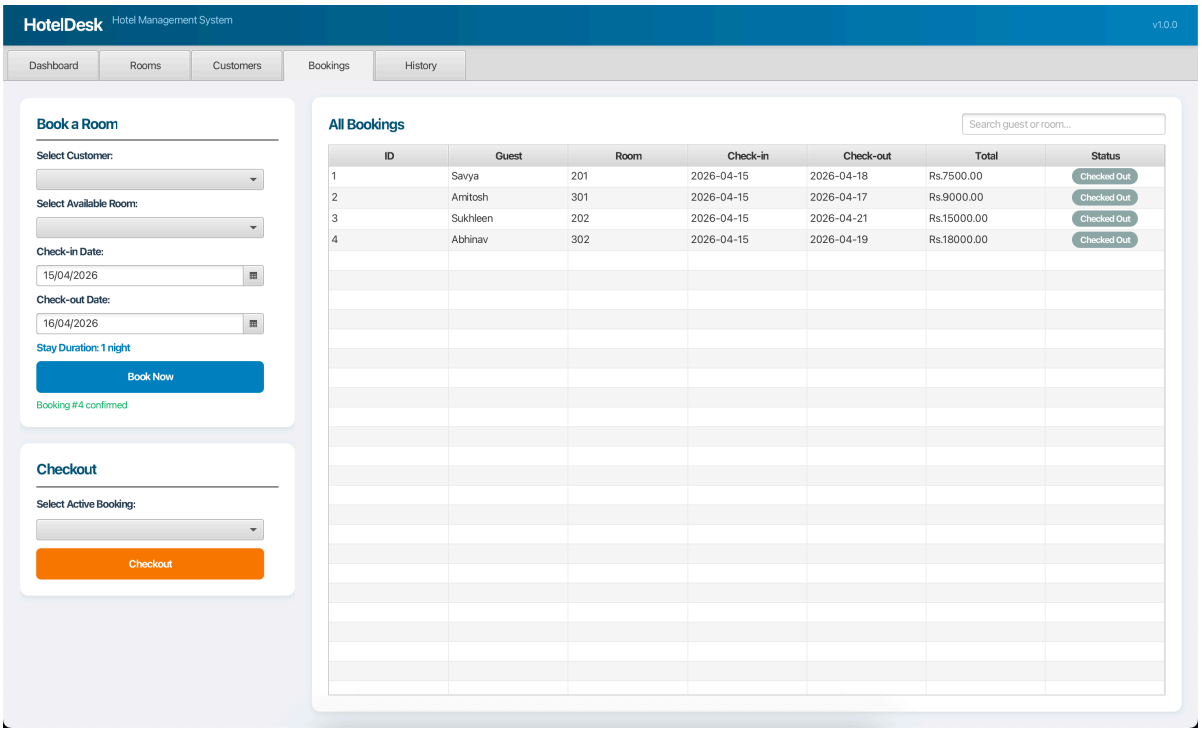


7.3 Button Styling

All action buttons are styled with inline -fx-* CSS strings. A reusable styleButton() helper method in BookingView accepts a Button and a hex colour string, applies a consistent look (280 px wide, 38 px tall, white text, bold 13 px, 6 px radius, hand cursor), and attaches mouse-enter / mouse-exit handlers to animate opacity between 1.0 and 0.85:

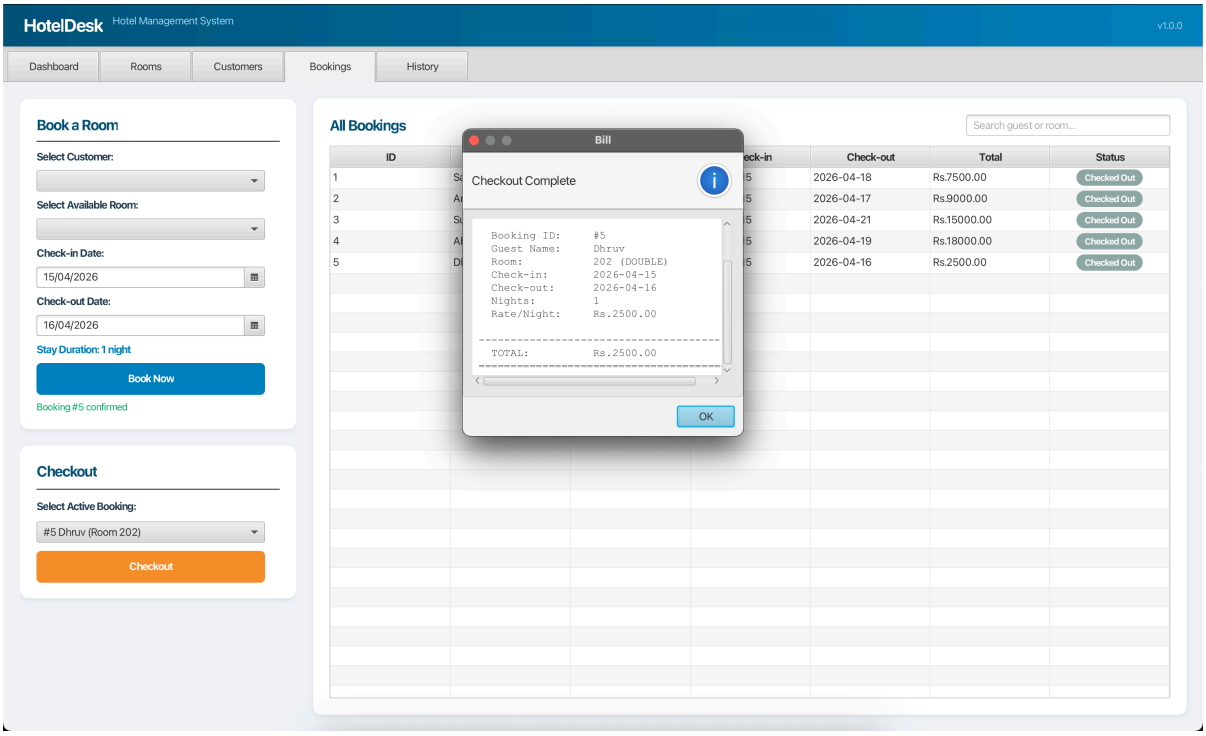
```
// BookingView.java
private void styleButton(Button btn, String color) {
    btn.setPrefWidth(280);
    btn.setPrefHeight(38);
    btn.setStyle("-fx-background-color: " + color + "; -fx-text-fill: white; "
            + "-fx-font-size: 13px; -fx-font-weight: bold; "
            + "-fx-background-radius: 6; -fx-cursor: hand;");
    btn.setOnMouseEntered(e -> btn.setOpacity(0.85));
    btn.setOnMouseExited(e  -> btn.setOpacity(1.0));
}
```

Book Now is styled #2980b9 (blue). Checkout is styled #e67e22 (orange). Save Room is #2980b9. Register Customer is #27ae60 (green). Filter buttons (Available Only = #8e44ad, View All = #2c3e50) use the same opacity hover pattern inline.



7.4 Input Validation

Each form uses a layered validation approach before committing data. RoomView and CustomerView show Alert.AlertType.ERROR dialogs for empty fields, non-numeric input, non-positive values, and duplicate room numbers. BookingView validates that both a customer and room are selected, that both DatePicker values are non-null, and that the check-out date is not before the check-in date (checkOutDate.isBefore(checkInDate)). The contact field enforces digits-only and maximum-10-character constraints live via a textProperty change listener.



7.5 History Tab

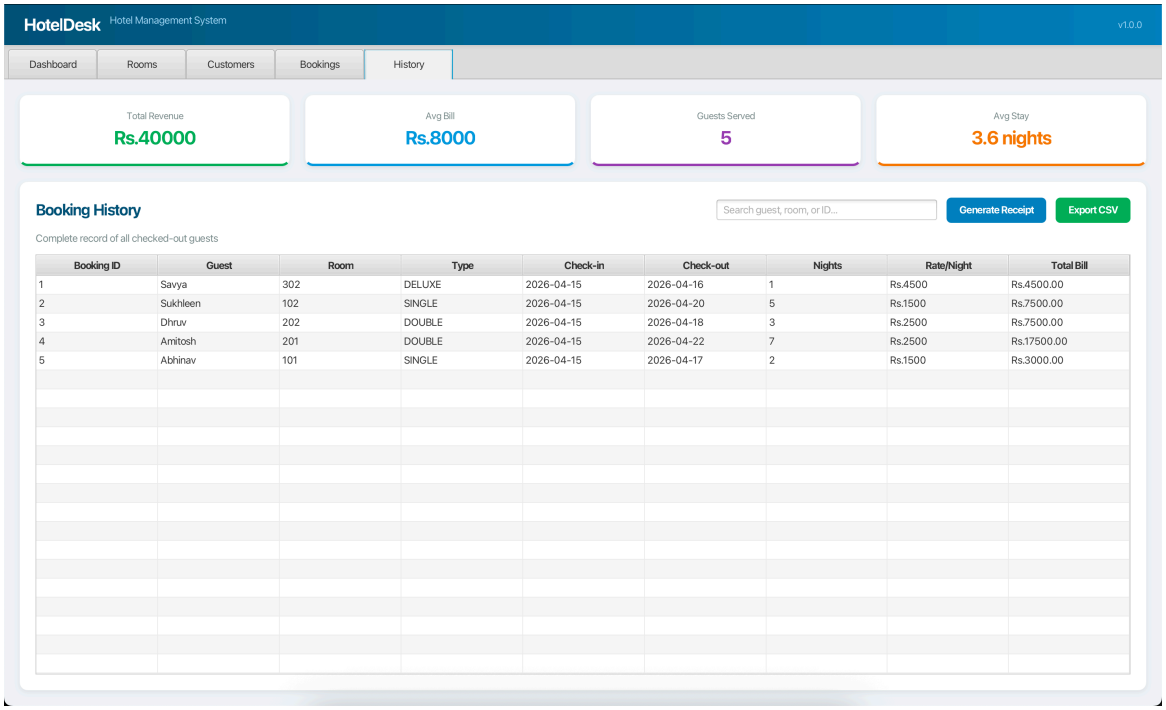
Analytics Cards. Four metric cards — Total Revenue, Average Bill, Guests Served, and Average Stay — are displayed at the top of the tab. These values are recalculated dynamically via the `updateStats()` method, which iterates over the filtered list and computes cumulative revenue, guest count, and average night duration using `ChronoUnit.DAYS.between(checkInDate, checkOutDate)`. The cards update automatically through a `ListChangeListener<Booking>` registered on the `BookingManager`'s observable list, ensuring real-time accuracy without any manual refresh.

History Table. A `TableView<Booking>` displays nine columns: Booking ID, Guest Name, Room Number, Room Type, Check-in Date, Check-out Date, Nights, Rate/Night, and Total Bill.

Search. A `TextField` above the table enables live filtering by guest name, booking ID, or room number. The search predicate is layered on top of the active/inactive filter, ensuring that only checked-out bookings matching the query are displayed. The analytics cards also refresh on every search input change.

Export CSV. The Export CSV button opens a `FileChooser` configured with a `.csv` extension filter and a default filename of `hotel_history.csv`. A `PrintWriter` iterates over the current `FilteredList` and writes a header row followed by comma-separated booking records. Success and failure are communicated via `Alert.INFORMATION` and `Alert.ERROR` dialogs respectively.

Receipt Generation. The Generate Receipt button reads the currently selected `TableView` row via `table.getSelectionModel().getSelectedItem()`. If no row is selected, a `Warning` alert prompts the user. On valid selection, a formatted plain-text invoice is constructed using `StringBuilder`, containing guest details, stay details (check-in, check-out, nights, rate), and the grand total. The receipt is displayed in a non-editable `TextArea` inside an `Alert` dialog. A nested "Save as Text File" button within the dialog opens a second `FileChooser`, allowing the user to persist the receipt as a `.txt` file to any location on disk.



Booking ID	Guest	Room	Type	Check-in	Check-out	Nights	Rate/Night	Total Bill
1	Savya	302	DELUXE	2026-04-15	2026-04-16	1	Rs.4500	Rs.4500.00
2	Sukhleen	102	SINGLE	2026-04-15	2026-04-20	5	Rs.1500	Rs.7500.00
3	Dhruv	202	DOUBLE	2026-04-15	2026-04-18	3	Rs.2500	Rs.7500.00
4	Amitosh	201	DOUBLE	2026-04-15	2026-04-22	7	Rs.2500	Rs.17500.00
5	Abhinav	101	SINGLE	2026-04-15	2026-04-17	2	Rs.1500	Rs.3000.00

8. Program Structure

```
HotelDesk/
│
├── pom.xml
│
└── src/
    └── main/
        └── java/
            ├── module-info.java
            │
            └── com/
                └── hotel/
                    └── hoteldesk/
                        │
                        ├── Main.java
                        │
                        ├── model/
                        │   ├── RoomType.java
                        │   ├── Room.java
                        │   ├── Customer.java
                        │   └── Booking.java
                        │
                        ├── manager/
                        │   ├── RoomManager.java
                        │   ├── CustomerManager.java
                        │   └── BookingManager.java
                        │
                        └── view/
                            ├── DashboardView.java
                            ├── RoomView.java
                            ├── CustomerView.java
                            ├── BookingView.java
                            └── HistoryView.java
```

9. Conclusion

HotelDesk successfully implements a complete desktop Hotel Management System using Java 25 and JavaFX 23.0.1, meeting all the stated objectives. The application is modular (JPMS), architecturally layered (model / manager / view), and fully reactive through the consistent use of observable collections with property extractors.

The most significant technical insight in the implementation is the use of ObservableList property extractors. By declaring room -> new Observable[]{room.availableProperty()} in RoomManager and customer -> new Observable[]{customer.assignedRoomProperty()} in CustomerManager, the application ensures that property mutations on existing list elements fire change events — keeping the Dashboard stat cards, the Room status badges, and the Customers Assigned Room column perpetually up to date without a single manual refresh call.

The inclusion of DatePicker controls in the booking workflow adds a meaningful layer of real-world practicality: guests are booked for explicit date ranges, and the bill is computed from the number of nights between checkInDate and plannedCheckOutDate using ChronoUnit.DAYS.between(), with a minimum of one night enforced.

Possible future enhancements include: persistent storage via SQLite or a flat-file JSON database; user authentication with role-based tab access; a reporting screen with JavaFX BarChart or PieChart; print-to-PDF for checkout bills; and a responsive layout for larger display sizes.

10. References

1. Oracle / OpenJDK. Java SE 25 Documentation. https://docs.oracle.com/en/java/javase/25/
2. OpenJFX Project. JavaFX 23 API Documentation. https://openjfx.io/javadoc/23/
3. OpenJFX. JavaFX Maven Plugin (javafx-maven-plugin) v0.0.8. https://github.com/openjfx/javafx-maven-plugin
4. Apache Maven Project. Maven Build Lifecycle Reference. https://maven.apache.org/guides/
5. Oracle. Java Platform Module System (JPMS) Guide. https://dev.java/learn/modules/
6. Bloch, J. (2018). Effective Java (3rd ed.). Addison-Wesley. Chapters on generics, lambdas, and effective use of the standard library.
7. Sharan, K. (2015). JavaFX 8: Introduction by Example. Apress. Observable properties, data binding, and TableView patterns.

11. Appendix (Code Snippets)

Key code extracts from the actual source files in HotelDesk-main, illustrating the central implementation patterns.

A1 module-info.java — JPMS Module Declaration

```
module com.hotel.hoteldesk {
    requires javafx.controls;
    opens com.hotel.hoteldesk to javafx.graphics;
    opens com.hotel.hoteldesk.model to javafx.base;
    exports com.hotel.hoteldesk;
}
```

A2 Room.java — BooleanProperty for Observable Availability

```
public class Room {
    private final int roomNumber;
    private final RoomType roomType;
    private final double pricePerDay;
    private final BooleanProperty available;

    public Room(int roomNumber, RoomType roomType, double pricePerDay) {
        this.roomNumber  = roomNumber;
        this.roomType    = roomType;
        this.pricePerDay = pricePerDay;
        this.available   = new SimpleBooleanProperty(true);
    }

    public boolean isAvailable()          { return available.get(); }
    public void setAvailable(boolean val) { available.set(val); }
    public BooleanProperty availableProperty() { return available; }
}
```

A3 RoomManager.java — ObservableList with Property Extractor & FilteredList

```
public RoomManager() {
    rooms = FXCollections.observableArrayList(
        room -> new Observable[]{ room.availableProperty() }
    );
    filteredRooms = new FilteredList<>(rooms, p -> true);
    loadSampleRooms();
}

public void setFilterAvailableOnly(boolean val) {
    if (val) {
        filteredRooms.setPredicate(Room::isAvailable);
    } else {
        filteredRooms.setPredicate(p -> true);
    }
}
```

A4 BookingManager.java — book() with LocalDate Parameters

```
public Booking book(Customer customer, Room room,
                    LocalDate checkIn, LocalDate checkOut) {
    if (!room.isAvailable()) return null;

    Booking booking = new Booking(
            nextBookingId++,
            customer.getCustomerId(), customer.getName(),
            room.getRoomNumber(), room.getRoomType(), room.getPricePerDay(),
            checkIn, checkOut
    );
    room.setAvailable(false);
    customer.setAssignedRoom(room.getRoomNumber());
    bookings.add(booking);
    return booking;
}
```

A5 BookingManager.java — checkout() with Bill String & bookings.set()

```
public String checkout(Booking booking) {
    if (!booking.isActive()) return null;
    booking.checkout();   // sets actualCheckOutDate = plannedCheckOutDate

    roomManager.findRoom(booking.getRoomNumber()).setAvailable(true);
    customerManager.findById(booking.getCustomerId()).setAssignedRoom(-1);
    totalRevenue += booking.getTotalAmount();

    // Replace element to fire a list-change event for the TableView
    int idx = bookings.indexOf(booking);
    if (idx >= 0) bookings.set(idx, booking);
    long nights = ChronoUnit.DAYS.between(
        booking.getCheckInDate(), booking.getCheckOutDate());
    if (nights < 1) nights = 1;
    return String.format(
        "===========================\n" +
        "    HOTEL BILL SUMMARY    \n" +
        "===========================\n" +
        "  Booking ID : #%d\n  Guest     : %s\n" +
        "  Room       : %d (%s)\n" +
        "  Check-in   : %s\n  Check-out : %s\n" +
        "  Nights     : %d\n  Rate/Night: Rs.%.2f\n" +
        "---------------------------\n" +
        "  TOTAL      : Rs.%.2f\n" +
        "===========================",
        booking.getBookingId(), booking.getCustomerName(),
        booking.getRoomNumber(), booking.getRoomType(),
        booking.getCheckInDate(), booking.getCheckOutDate(),
        nights, booking.getPricePerDay(), booking.getTotalAmount());
}
```

A6 BookingView.java — DatePicker Booking Flow

```
DatePicker checkInPicker  = new DatePicker();
checkInPicker.setValue(LocalDate.now());
DatePicker checkOutPicker = new DatePicker();
```

```
checkOutPicker.setValue(LocalDate.now().plusDays(1));

bookBtn.setOnAction(e -> {
    LocalDate checkInDate  = checkInPicker.getValue();
    LocalDate checkOutDate = checkOutPicker.getValue();

    if (checkOutDate.isBefore(checkInDate)) {
        showAlert(Alert.AlertType.ERROR, "Invalid Dates",
                  "Check-out cannot be before check-in.");
        return;
    }
    Booking booking = bookingManager.book(
            customer, room, checkInDate, checkOutDate);
    ...
});
```

A7 BookingView.java — refreshActiveBookings() with FilteredList

```
private void refreshActiveBookings() {
    FilteredList<Booking> activeBookings =
            bookingManager.getBookings().filtered(Booking::isActive);
    bookingCombo.getItems().setAll(activeBookings);
}
```

A8 DashboardView.java — ListChangeListeners & refreshDashboard()

```
roomManager.getRooms().addListener(
    (ListChangeListener<Room>) c -> refreshDashboard());
bookingManager.getBookings().addListener(
    (ListChangeListener<Booking>) c -> refreshDashboard());

private void refreshDashboard() {
    totalRoomsValue   .setText(String.valueOf(roomManager.getTotalRooms()));

availableRoomsValue.setText(String.valueOf(roomManager.getAvailableRooms()));
    occupiedRoomsValue .setText(String.valueOf(roomManager.getOccupiedRooms()));

activeBookingsValue.setText(String.valueOf(bookingManager.getActiveBookings()));
    totalRevenueValue  .setText(String.format("Rs.%.0f",
                               bookingManager.getTotalRevenue()));
}
```

End of Report